

✓ Formative Assignment: Advanced Linear Algebra (PCA)

This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

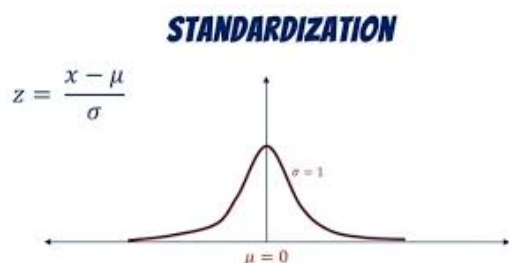
1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

```
from google.colab import drive
drive.mount('/content/drive')
```

✓ Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below



```
import numpy as np

# Load dataset
data = np.genfromtxt('/content/african_crises.csv',
                    delimiter=',',
                    names=True,
                    dtype=None,
                    encoding='utf-8')

# Select numerical columns only
X = np.column_stack((
    data['year'],
    data['systemic_crisis'],
    data['exch_usd'],
    data['domestic_debt_in_default'],
    data['sovereign_external_debt_default'],
    data['gdp_weighted_default'],
    data['inflation_annual_cpi'],
    data['independence'],
    data['currency_crises'],
```

```

data['inflation_crises']
))

# Standardization
mean = np.mean(X, axis=0)
std = np.std(X, axis=0)

standardized_data = (X - mean) / std

standardized_data[:5]

array([[ -2.91715048,  3.45175812, -0.38671252, -0.20321893, -0.42497295,
        -0.14700624, -0.03086348, -1.86235185, -0.37805817, -0.38547376],
       [ -2.88731291, -0.28970744, -0.38670773, -0.20321893, -0.42497295,
        -0.14700624, -0.03084763, -1.86235185, -0.37805817, -0.38547376],
       [ -2.85747534, -0.28970744, -0.38671243, -0.20321893, -0.42497295,
        -0.14700624, -0.03087408, -1.86235185, -0.37805817, -0.38547376],
       [ -2.82763778, -0.28970744, -0.38671776, -0.20321893, -0.42497295,
        -0.14700624, -0.03085199, -1.86235185, -0.37805817, -0.38547376],
       [ -2.79780021, -0.28970744, -0.3867211 , -0.20321893, -0.42497295,
        -0.14700624, -0.03087427, -1.86235185, -0.37805817, -0.38547376]])

```

✓ Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```

cov_matrix = np.cov(standardized_data, rowvar=False)

cov_matrix

array([[ 1.00094518,  0.19763708,  0.2489921 ,  0.13695733,  0.27214712,
        -0.05472206,  0.03706992,  0.40774521,  0.18956951,  0.09872358],
       [ 0.19763708,  1.00094518,  0.20287813,  0.12227396,  0.25008637,
        0.00527918,  0.10655226,  0.14722222,  0.11285756,  0.17272494],
       [ 0.2489921 ,  0.20287813,  1.00094518,  0.005258 ,  0.42328974,
        -0.04076417, -0.01195788,  0.12615328, -0.0565252 , -0.06384318],
       [ 0.13695733,  0.12227396,  0.005258 ,  1.00094518,  0.46519012,
        -0.02990269,  0.15197534,  0.10922265,  0.22780017,  0.22464067],
       [ 0.27214712,  0.25008637,  0.42328974,  0.46519012,  1.00094518,
        0.34624602,  0.07267715,  0.22840723,  0.19961649,  0.18810757],
       [-0.05472206,  0.00527918, -0.04076417, -0.02990269,  0.34624602,
        1.00094518, -0.00453969,  0.07901041,  0.01698597,  0.0176471 ],
       [ 0.03706992,  0.10655226, -0.01195788,  0.15197534,  0.07267715,
        -0.00453969,  1.00094518,  0.01658431,  0.07666228,  0.08013589],
       [ 0.40774521,  0.14722222,  0.12615328,  0.10922265,  0.22840723,
        0.07901041,  0.01658431,  1.00094518,  0.08645776, -0.02256886],
       [ 0.18956951,  0.11285756, -0.0565252 ,  0.22780017,  0.19961649,
        0.01698597,  0.07666228,  0.08645776,  1.00094518,  0.3937481 ],
       [ 0.09872358,  0.17272494, -0.06384318,  0.22464067,  0.18810757,
        0.0176471 ,  0.08013589, -0.02256886,  0.3937481 ,  1.00094518]])

```

In a paragraph that has a maximum of 5 Lines, Explain why we need to compute a covariance matrix, provide at least 2 reasons

✓ Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

eigenvalues, eigenvectors

(array([2.38749792, 1.45209083, 0.25485013, 1.17824746, 0.52751592,
        0.57998809, 0.74308878, 1.03364791, 0.96177261, 0.89075214]),
 array([[ 0.37861403,  0.24364849, -0.04139686,  0.38775075,  0.69294326,
          0.19174338, -0.19741544, -0.27495547, -0.11687899,  0.00724834],
        [ 0.3187303 ,  0.06220911, -0.02931357,  0.15277511,  0.09342902,
         -0.20386773,  0.57722192,  0.29605001,  0.19006889, -0.60257243],
        [ 0.26821151,  0.48735862, -0.40611346,  0.01701645, -0.26893457,
         -0.01306867, -0.42929652,  0.34784167,  0.37964572,  0.05839231],
        [ 0.36661898, -0.26519236, -0.42033643, -0.08221407,  0.08246465,
         -0.02093874,  0.40145015,  0.19165558, -0.12467699,  0.62541537],
        [ 0.50514042,  0.11726494,  0.71662774, -0.39531132, -0.00368752,
         -0.00758231, -0.03923176,  0.11119683,  0.09792331,  0.19414378],
        [ 0.12076417,  0.05447335, -0.37021982, -0.7757479 ,  0.17317552,
          0.0170927 , -0.06097553, -0.27098874, -0.16298578, -0.33106187],
        [ 0.13488825, -0.20624654,  0.02346395,  0.06215209, -0.0360556 ,
          0.03431472, -0.3388281 ,  0.54810637, -0.68354568, -0.22342068],
        [ 0.30475813,  0.3135348 , -0.01808484,  0.1887567 , -0.59195487,
          0.06931768,  0.20102859, -0.44108157, -0.42624884, -0.03151539],
        [ 0.30528124, -0.45034029, -0.04666739,  0.13440997, -0.09233895,
         -0.67420928, -0.33420735, -0.29939313,  0.12396016, -0.06302015],
        [ 0.27330514, -0.51822151, -0.04628894,  0.06038676, -0.20428906,
          0.67836049, -0.10107225, -0.0928373 ,  0.30015938, -0.20031113]]))
```

✓ Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

[How Is Explained Variance Used In PCA?](#)

```
sorted_indices = np.argsort(eigenvalues)[::-1]

sorted_eigenvectors = eigenvectors[:, sorted_indices]

sorted_eigenvectors

array([[ 0.37861403,  0.24364849,  0.38775075, -0.27495547, -0.11687899,
          0.00724834, -0.19741544,  0.19174338,  0.69294326, -0.04139686],
        [ 0.3187303 ,  0.06220911,  0.15277511,  0.29605001,  0.19006889,
         -0.60257243,  0.57722192, -0.20386773,  0.09342902, -0.02931357],
        [ 0.26821151,  0.48735862,  0.01701645,  0.34784167,  0.37964572,
          0.05839231, -0.42929652, -0.01306867, -0.26893457, -0.40611346],
        [ 0.36661898, -0.26519236, -0.08221407,  0.19165558, -0.12467699,
          0.62541537,  0.40145015, -0.02093874,  0.08246465, -0.42033643],
        [ 0.50514042,  0.11726494, -0.39531132,  0.11119683,  0.09792331,
          0.19414378, -0.03923176, -0.00758231, -0.00368752,  0.71662774],
        [ 0.12076417,  0.05447335, -0.7757479 , -0.27098874, -0.16298578,
         -0.33106187, -0.06097553,  0.0170927 ,  0.17317552, -0.37021982],
```

```
[ 0.13488825, -0.20624654, 0.06215209, 0.54810637, -0.68354568,
 -0.22342068, -0.3388281 , 0.03431472, -0.0360556 , 0.02346395],
 [ 0.30475813, 0.3135348 , 0.1887567 , -0.44108157, -0.42624884,
 -0.03151539, 0.20102859, 0.06931768, -0.59195487, -0.01808484],
 [ 0.30528124, -0.45034029, 0.13440997, -0.29939313, 0.12396016,
 -0.06302015, -0.33420735, -0.67420928, -0.09233895, -0.04666739],
 [ 0.27330514, -0.51822151, 0.06038676, -0.0928373 , 0.30015938,
 -0.20031113, -0.10107225, 0.67836049, -0.20428906, -0.04628894]])
```

✓ Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
# Keep first 2 principal components
num_components = 2

principal_components = sorted_eigenvectors[:, :num_components]

reduced_data = np.dot(standardized_data, principal_components)

reduced_data[:5]

array([[ -1.20743899, -0.89597745],
       [ -2.38865709, -1.12146175],
       [ -2.377365 , -1.1141887 ],
       [ -2.36606652, -1.10692598],
       [ -2.3547735 , -1.09965313]])
```

✓ Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
print(f"Reduced Data Shape: {reduced_data.shape}")

reduced_data[:5]

Reduced Data Shape: (1059, 2)
array([[ -1.20743899, -0.89597745],
       [ -2.38865709, -1.12146175],
       [ -2.377365 , -1.1141887 ],
       [ -2.36606652, -1.10692598],
       [ -2.3547735 , -1.09965313]])
```

✓ Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

```
import matplotlib.pyplot as plt

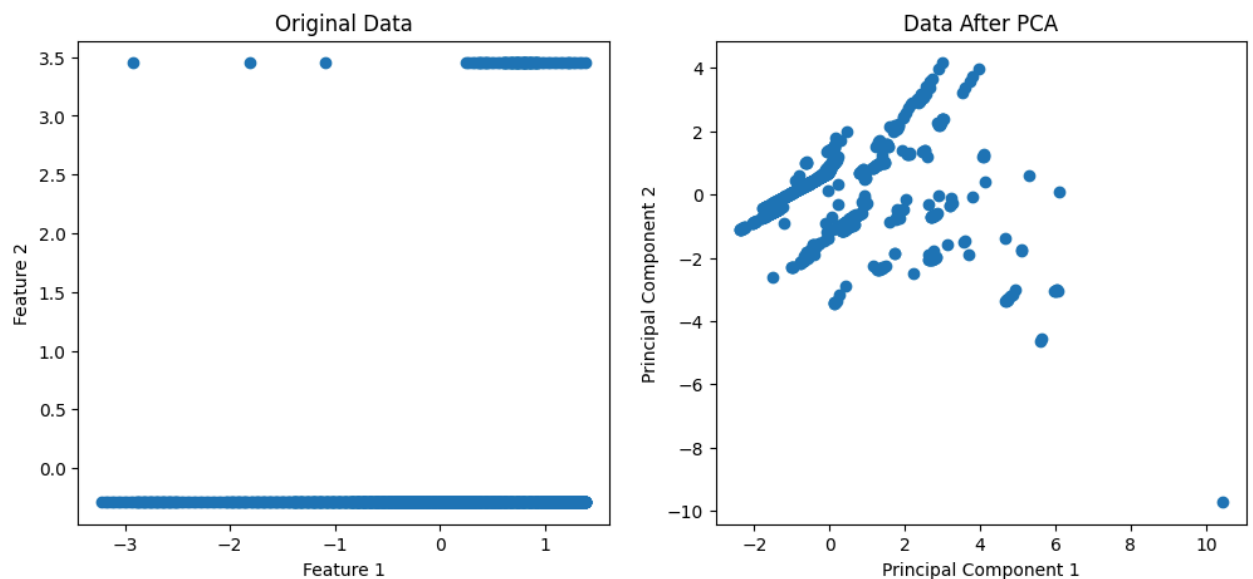
# Original Data
```

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.scatter(
    standardized_data[:,0],
    standardized_data[:,1]
)
plt.title("Original Data")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")

# PCA Data
plt.subplot(1,2,2)
plt.scatter(
    reduced_data[:,0],
    reduced_data[:,1]
)
plt.title("Data After PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.show()
```



Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA

In the original plot, the data is distributed over several correlated features, and it is hard to recognize patterns in the data. The data is then converted to principal components that represent the largest amount of variance after The PCA. The reduced plot seems more orderly in nature and emphasizes the major features of the data set. This will help to see relationships between observations. PCA reduces the data without losing a lot of the significant information.

- 2.Explain why you selected the number of principle components, more especially explain what tradeoffs you are making.

To visualize in a two dimensional space I chose two principal components that explain a significant portion of the variance in the data set. This selection will simplify and minimize complexity and computation. Enhancement is associated with the loss of some variance in the components that are left. To retain more components would retain more information, but at the cost of the dimensionality reduction benefits.

3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

Dimensionality reduction may remove smaller variations related to economic activity, such as inflation, exchange rates, and debt defaults. Information about population pressure may also be compressed or lost. Some weak relationships between variables are not fully preserved. The major patterns remain visible in the reduced dataset. However, some detailed country-specific information is sacrificed.